Kubernetes: основные понятия и архитектура

Занятие 01 · День 1, утро

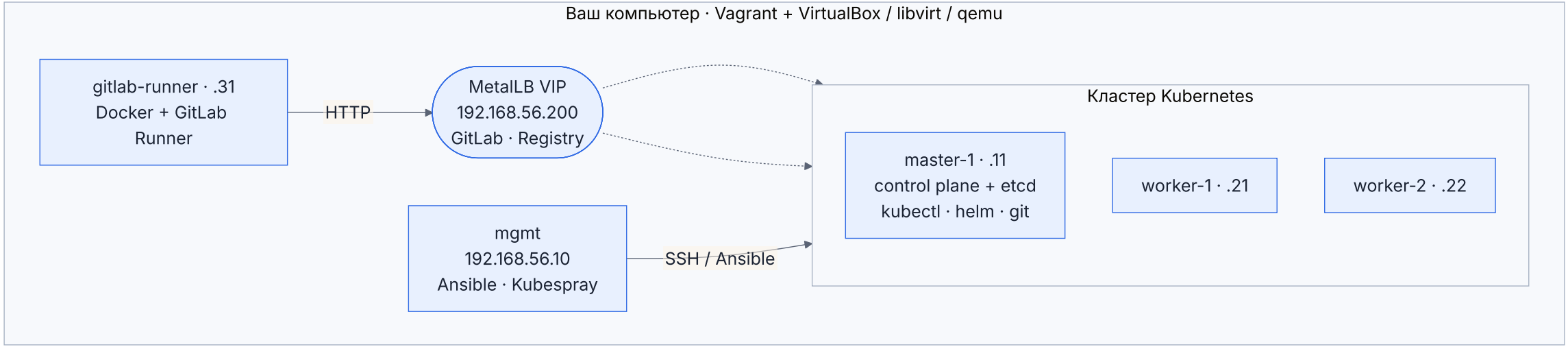
DevOps продвинутый курс: Kubernetes и CI/CD на основе GitLab

План курса — 5 дней

День	Утро	После обеда
1	**01** Kubernetes: основы и архитектура	**02** Виды установок, Kubespray
2	**03** Основные сущности	**04** YAML-манифесты
3	**05** Первое приложение	**06** Основы Git
4	**07** CI/CD, обзор GitLab, GitLab в k8s	**08** Первый проект · **09** GitLab Runner
5	**10** `.gitlab-ci.yml`, best practices, Helm	**11** Интеграция GitLab с Kubernetes

Каждый блок: **теория + демонстрация** → **практическое задание** на вашем стенде.

Учебный стенд



- Всё поднимается одной командой `vagrant up` на вашем компьютере
- ОС — **Ubuntu 24.04**, провайдеры: VirtualBox, libvirt, qemu

Что получим к концу курса

- Кластер Kubernetes, **собранный вами** с помощью Kubespray
- Приложение, описанное манифестами и упакованное в **Helm-чарт**
- **GitLab**, развёрнутый внутри кластера
- **GitLab Runner** на отдельной машине
- Пайплайн: `git push` → тесты → сборка образа → **деплой в Kubernetes**

> Цель — не «нажать кнопку», а понимать, что происходит на каждом шаге.

Зачем нужен Kubernetes

Контейнеры, оркестрация, декларативный подход

Эволюция развёртывания

Эпоха	Как запускали	Боль
Железные серверы	приложение прямо на ОС	конфликты зависимостей, простои при обновлении
Виртуальные машины	одна ВМ — одно приложение	тяжело, медленно, дорого
Контейнеры	образ + runtime (Docker, containerd)	десятки контейнеров на десятках серверов — кто ими управляет?
Оркестрация	Kubernetes	— сложность самого Kubernetes

Контейнер за один слайд

- **Образ** — неизменяемый набор слоёв файловой системы + метаданные (OCI)
- **Контейнер** — обычный процесс Linux, изолированный ядром:
  - **namespaces** — что процесс *видит*: pid, net, mnt, uts, ipc, user
  - **cgroups** — сколько процесс *может потребить*: CPU, память, I/O
- **Runtime**: `containerd` / `CRI-O` → `runc`

```
docker run -d --name web -p 8080:80 nginx:1.28-alpine
ps aux | grep nginx          # на хосте это просто процесс
```

Что должен уметь оркестратор

- **Размещать** контейнеры на серверах с учётом ресурсов
- **Перезапускать** упавшие, переносить с упавшего узла
- **Масштабировать** — вручную и автоматически
- **Обновлять без простоя** и откатывать
- Давать **стабильный адрес** набору контейнеров (сервис, балансировка, DNS)
- Хранить **конфигурацию и секреты** отдельно от образа
- Подключать **хранилище** к контейнерам

Kubernetes делает всё это — и предоставляет единый **API**.

Kubernetes в фактах

- Создан в Google по опыту внутренней системы **Borg**, открыт в **2014**
- Передан в **CNCF** (Cloud Native Computing Foundation)
- Название — «рулевой» (греч.), сокращение **K8s**
- Новая минорная версия **3 раза в год**, поддержка — ~14 месяцев
- Kubespray v2.32 по умолчанию ставит **Kubernetes 1.36**
- Вокруг — огромная экосистема: Helm, Prometheus, Argo CD, Flux, Istio, Cilium…

Императивно vs декларативно

Императивно — «сделай»

```
kubectl create deployment web \
  --image=nginx:1.28-alpine
kubectl scale deployment web \
  --replicas=3
```

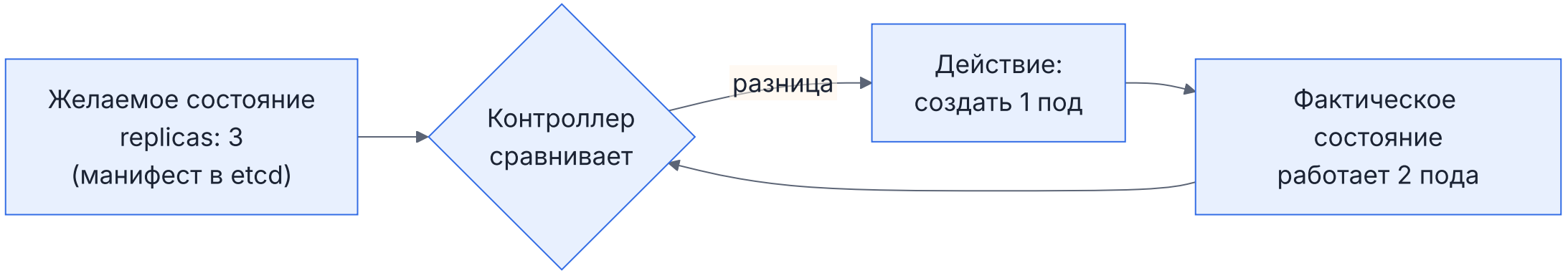
Что будет, если команду запустить дважды?

Декларативно — «хочу так»

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  ...
```

`kubectl apply -f web.yaml` — сколько угодно раз, результат один

Цикл согласования (reconciliation loop)

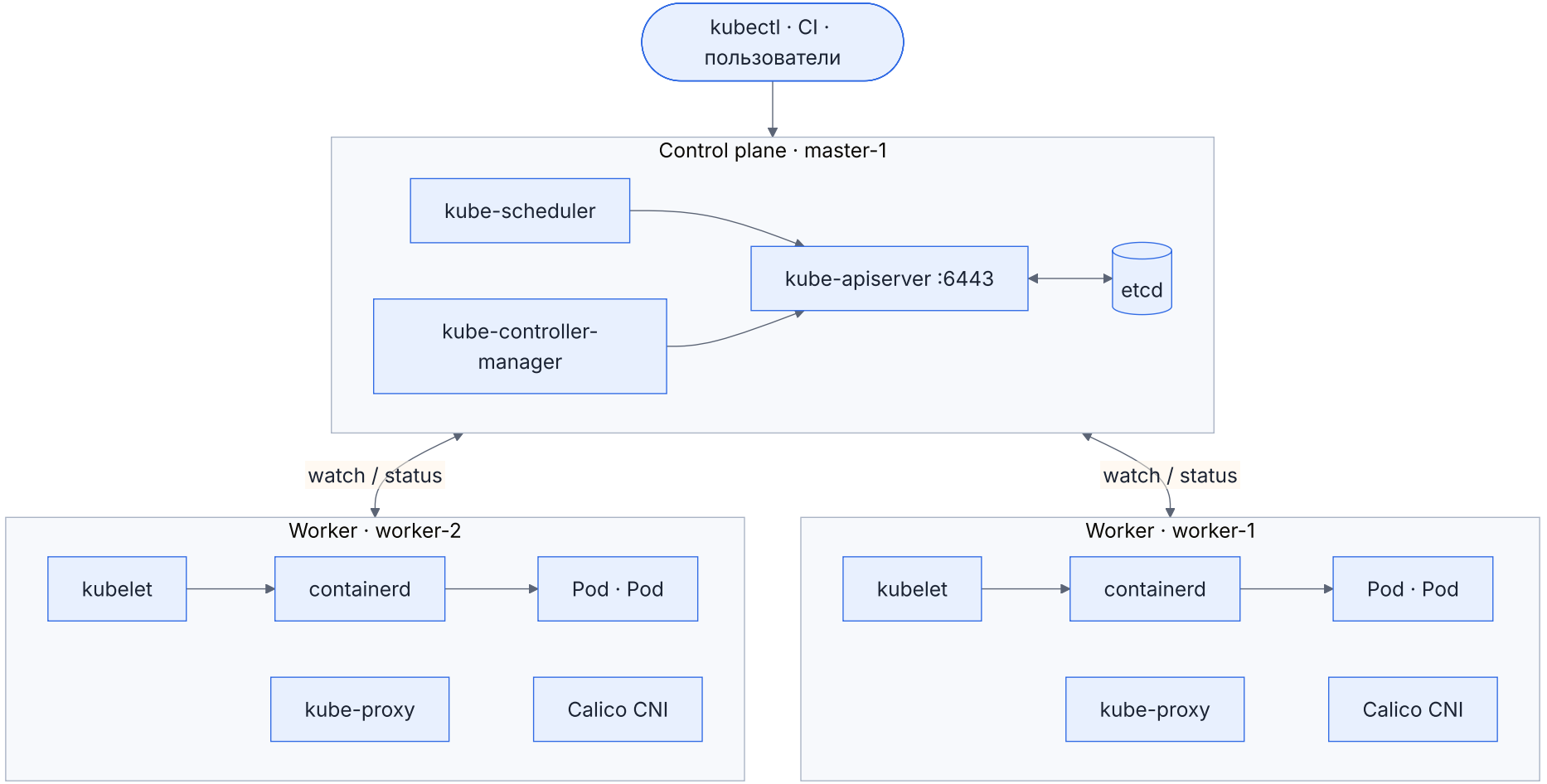


- Вы описываете **желаемое состояние** (`spec`)
- Контроллеры **постоянно** приводят к нему фактическое (`status`)
- Удалили под руками — контроллер создаст новый. Это **не баг, а суть** Kubernetes

Архитектура кластера

Control plane и рабочие узлы

Общая схема


kubectl · CI · пользователи
Control plane · master-1
kube-scheduler
kube-apiserver :6443
etcd
kube-controller-manager
watch / status
watch / status
Worker · worker-2
kubelet
containerd
Pod · Pod
kube-proxy
Calico CNI
Worker · worker-1
kubelet
containerd
Pod · Pod
kube-proxy
Calico CNI

kube-apiserver — сердце кластера

- **Единственный** компонент, который читает и пишет в etcd
- REST API: все — `kubectl`, kubelet, контроллеры, CI — ходят только сюда
- Цепочка обработки запроса:
  i. **Аутентификация** — кто вы? (сертификат, токен, OIDC)
  ii. **Авторизация** — можно ли? (RBAC)
  iii. **Admission** — изменить/проверить объект (mutating → validating)
  iv. Запись в **etcd**
- Слушает порт **6443**; масштабируется горизонтально (stateless)

etcd — память кластера

- Распределённое key-value хранилище, алгоритм консенсуса **Raft**
- Хранит **все объекты** кластера: `/registry/deployments/default/web`
- Запись подтверждается **кворумом** — большинством членов: ⌊N/2⌋+1

Членов etcd	Кворум	Переживёт отказ
1	1	0
3	2	**1**
5	3	**2**

> Кворум — основа Raft, его нельзя «отключить». Чётное число членов не добавляет надёжности. Бэкап etcd = бэкап кластера.

kube-scheduler

Решает **на каком узле** запустить под (сам под не запускает!)

1. **Фильтрация** — отбросить узлы, которые не подходят:
   хватает ли CPU/памяти (`requests`), `nodeSelector`, taints/tolerations, affinity
2. **Оценка (scoring)** — выбрать лучший из оставшихся:
   равномерность нагрузки, распределение реплик по узлам
3. **Привязка** — записать `spec.nodeName` в объект Pod через API

Нет подходящего узла → под остаётся в **Pending**, в событиях — причина.

kube-controller-manager

Набор контроллеров в одном процессе — каждый следит за своим типом объектов:

Контроллер	Что делает
Deployment	создаёт/обновляет ReplicaSet
ReplicaSet	держит нужное число подов
Node	следит за доступностью узлов, выселяет поды
Job / CronJob	запускает задачи до завершения / по расписанию
EndpointSlice	связывает Service с IP подов
ServiceAccount	создаёт SA `default` в новых namespace

cloud-controller-manager — то же для облачных ресурсов (LB, диски). У нас его роль частично выполняет **MetalLB**.

Рабочий узел

- **kubelet** — агент на каждом узле: получает поды, назначенные на узел, и запускает их через CRI; выполняет пробы; отчитывается о статусе
- **Container runtime** (CRI) — `containerd` у нас. *dockershim удалён в 1.24* — Docker как runtime узла больше не используется, но образы Docker работают
- **kube-proxy** — реализует Service: правила `iptables` / `ipvs` / `nftables` *(Kubespray по умолчанию — ipvs)*
- **CNI-плагин** — сеть подов: Calico, Cilium, Flannel… *(у нас — Calico)*

Как запускается сам control plane?

Static Pods — kubelet на master-узле сам запускает поды из файлов:

```
vagrant@master-1:~$ ls /etc/kubernetes/manifests/
kube-apiserver.yaml  kube-controller-manager.yaml  kube-scheduler.yaml
```

- Не нужен работающий API, чтобы поднять API — проблема «курицы и яйца» решена
- В API видны как *mirror pods* (`kube-apiserver-master-1`), но удалить через kubectl нельзя
- В Kubespray **etcd** по умолчанию работает как **systemd-сервис**, а не static pod

Аддоны кластера

Аддон	Зачем
CoreDNS	DNS внутри кластера: `shop-api.shop.svc.cluster.local`
nodelocaldns	кэширующий DNS на каждом узле (Kubespray)
Calico	сеть подов, NetworkPolicy
metrics-server	`kubectl top`, автомасштабирование HPA
local-path-provisioner	тома (PVC) на локальном диске узла
MetalLB	`Service type: LoadBalancer` без облака

Все они — обычные поды в namespace `kube-system`, `metallb-system`, …

Что происходит при `kubectl apply`

1. `kubectl` отправляет Deployment в **apiserver** → authn → authz → admission → **etcd**
2. **Deployment controller** видит новый объект → создаёт **ReplicaSet**
3. **ReplicaSet controller** создаёт 3 объекта **Pod** (пока без узла)
4. **Scheduler** выбирает узел каждому поду → `spec.nodeName`
5. **kubelet** нужного узла видит «свой» под → **containerd** скачивает образ и запускает контейнер
6. **CNI** выдаёт поду IP, kubelet пишет статус `Running` обратно в API

Все компоненты общаются **только через apiserver** и механизм *watch*.

Сетевая модель Kubernetes

- Каждый **под получает свой IP**; все поды видят друг друга **без NAT**
- Контейнеры внутри пода делят сеть: общаются через `localhost`
- **Service** — стабильный виртуальный IP и DNS-имя перед набором подов

Диапазон (Kubespray по умолчанию)	Назначение
`10.233.64.0/18`	IP подов (`kube_pods_subnet`)
`10.233.0.0/18`	IP сервисов (`kube_service_addresses`)
`192.168.56.0/24`	IP узлов (наш стенд)

Kubernetes API: объекты

- У каждого объекта: `apiVersion`, `kind`, `metadata`, `spec`, `status`
- API разбит на **группы** и **версии**: `v1` (core), `apps/v1`, `batch/v1`, `networking.k8s.io/v1`, `gateway.networking.k8s.io/v1`
- Расширяется через **CRD** — свои типы объектов (так работают MetalLB, Calico, Gateway API)

```
kubectl api-resources | head
kubectl api-versions
kubectl explain deployment.spec.strategy
```

ДЕМО Смотрим на живой кластер

```
kubectl get nodes -o wide
kubectl get pods -n kube-system -o wide
kubectl get --raw='/readyz?verbose' | tail -5

# на master-1
sudo ls /etc/kubernetes/manifests
sudo crictl ps                          # контейнеры на узле через CRI
systemctl status kubelet etcd --no-pager

# что лежит в etcd
sudo etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/ssl/etcd/ssl/ca.pem \
  --cert=/etc/ssl/etcd/ssl/admin-master-1.pem \
  --key=/etc/ssl/etcd/ssl/admin-master-1-key.pem \
  get /registry/namespaces --prefix --keys-only
```

Демо — на заранее собранном кластере преподавателя; ваш кластер появится после обеда.

Где ещё живёт Kubernetes

Вариант	Примеры	Кто управляет control plane
Managed	EKS, GKE, AKS, Yandex Managed K8s	облако
Платформы	OpenShift, Rancher (RKE2), Deckhouse	вендор + вы
«Ванильный»	kubeadm, **Kubespray**	вы
Лёгкие	k3s, k0s, MicroK8s	вы
Локальные для разработки	kind, minikube, k3d	вы, на ноутбуке

Знание «ванильного» Kubernetes переносится на все варианты.

Итоги

- Kubernetes — **декларативная** платформа: вы описываете цель, контроллеры её достигают
- **Control plane**: apiserver (единая точка входа), etcd (состояние, кворум), scheduler (куда), controller-manager (циклы согласования)
- **Узел**: kubelet + containerd + kube-proxy + CNI
- Всё общается **только через API**

Вопросы для самопроверки

1. Почему 3 члена etcd лучше, чем 4?
2. Под в статусе `Pending` — какой компонент «виноват» скорее всего?
3. Как kubelet запускает apiserver, если apiserver ещё не работает?

Практика 01

Разворачиваем учебный стенд

Задание — на платформе testlms.kz